

Análise de Algoritmos

Counting Sort

Avaliação de complexidade do pior caso e melhor caso



Docente:

Edson Castro

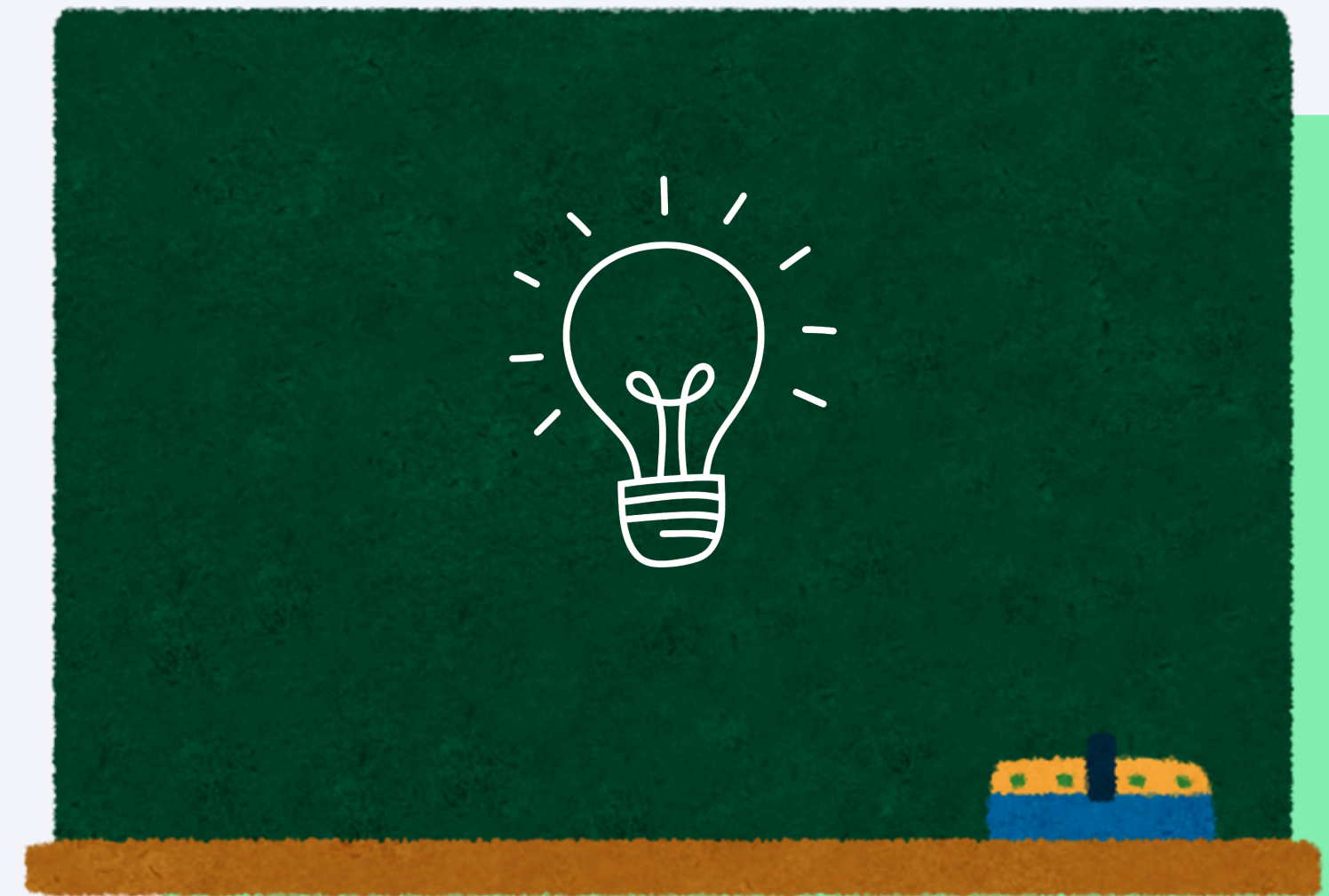
Discentes:

Cleiton Guilhermite
Leonardo Armelin

Introdução

O **Counting Sort** é um algoritmo de ordenação não baseado em comparações, possui uma implementação relativamente simples e é especialmente eficiente em casos onde o vetor de entrada é grande porém com um domínio pequeno.

Esse algoritmo pode ser utilizado de forma eficiente para ordenação de caracteres, contagem de frequência ou equalização de histogramas.



Características do Counting Sort

performance, aplicações e definições



**complexidade
temporal linear**



**Estabilidade do
Algoritmo**



**Uso de memória
auxiliar**



**Aplicável a dados
discretos e inteiros**



**Ordenação não
comparativa**



**Uso dependente do
“intervalo de valores”**



Trabalhando com o Counting Sort

Step by step

01

Identificação do Valor
Máximo no Vetor de Entrada

02

Inicialização do Vetor de
Contagem

03

Contagem das Ocorrências
dos Elementos

04

Cálculo da Soma Cumulativa
(Prefix Sum)

05

Posicionamento dos
Elementos no Vetor
Resultado

06

Atualização do Vetor de
Contagem Após Inserção

01 | Máximo

Encontramos o elemento máximo do vetor[].

	0	1	2	3	4	5	6	7	Max
arr[] =	2	5	3	0	2	3	0	3	5

Sumário:

- arr[]: vetor

- ctnArr[]: vetor contagem

- ans[]: resultado

02 Inicialização

Inicializamos um vetor_contagem[] de comprimento max+1 com todos os elementos sendo 0. Este vetor será utilizado para armazenar as ocorrências dos elementos do vetor de entrada.

	0	1	2	3	4	5
cntArr[] =	0	0	0	0	0	0

03

Contagem

No vetor `vetor_contagem[]`, armazenamos a contagem de cada elemento único do vetor de entrada em seus respectivos índices.

	0	1	2	3	4	5
<code>cntArr[]</code> =	2	0	2	3	0	1

04

Soma cumulativa

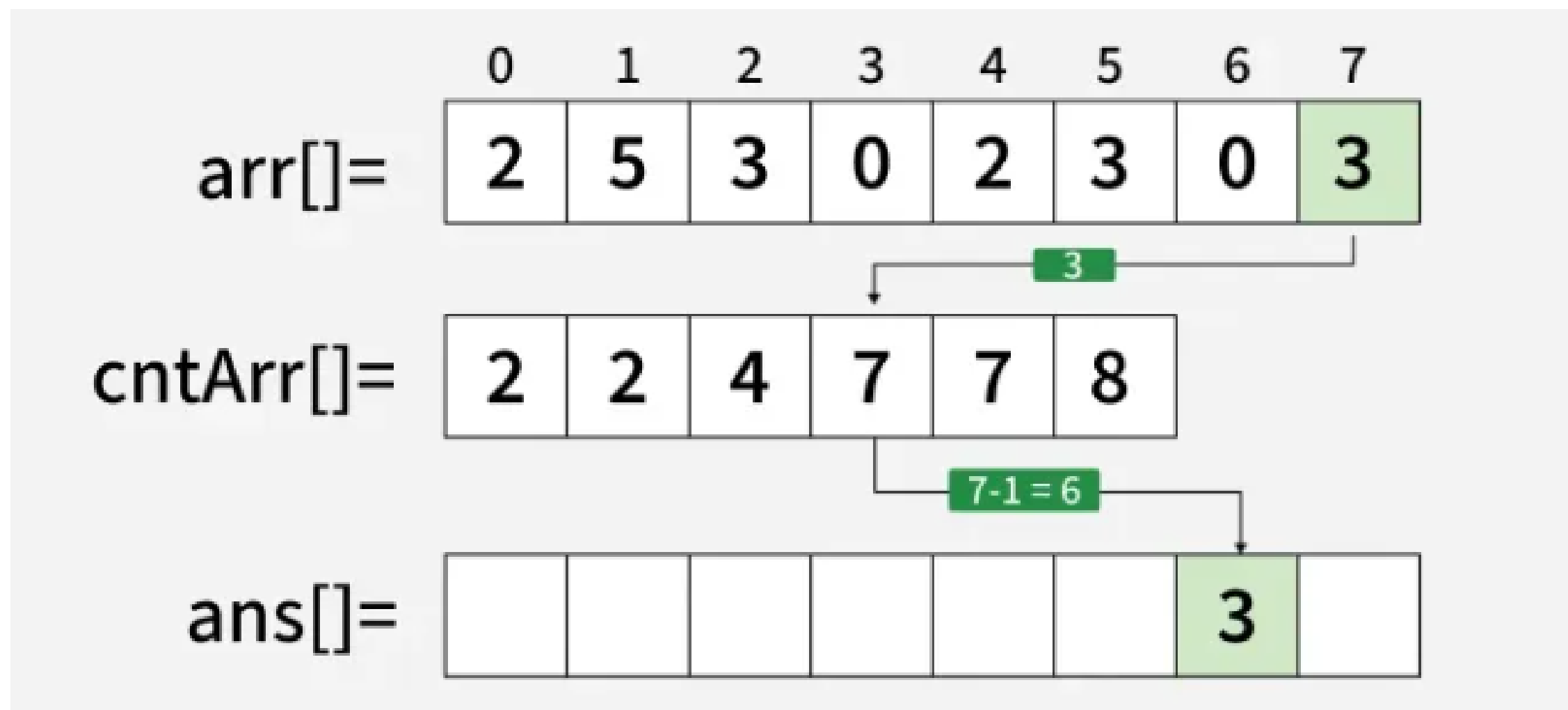
Armazenamos a soma cumulativa ou a soma prefixada dos elementos do vetor `vetor_contagem[]`, fazendo `vetor_contagem[i]=vetor_contagem[i-1]+vetor_contagem[i]`.

	0	1	2	3	4	5
cntArr[]=	2	2	4	7	7	8

05

Posicionamento de elementos

Atualize $\text{resultado}[\text{vetor_contagem}[\text{vetor}[i]] - 1] = \text{vetor}[i]$ e decrémente $\text{vetor_contagem}[\text{vetor}[i]]$.
Percorra o vetor de entrada na ordem inversa para manter a ordem dos elementos iguais, garantindo que a ordenação permaneça estável.

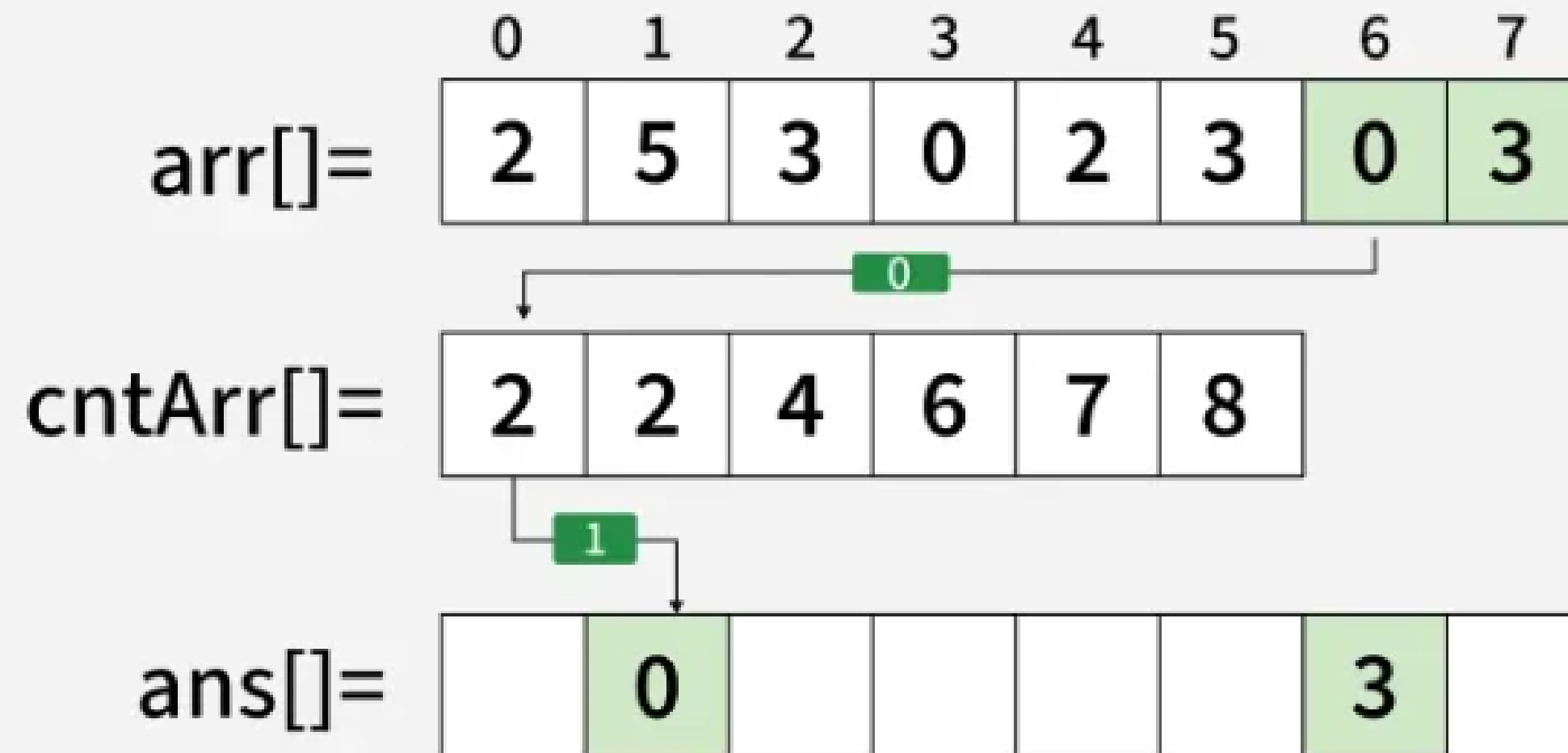


06

Atualizações

Atualizamos resultado $[\text{vetor_contagem} [\text{vetor} [6]] - 1] = \text{vetor} [6]$

também atualizamos $\text{vetor_contagem} [\text{vetor} [6]] = \text{vetor_contagem} [\text{vetor} [6]] --$.

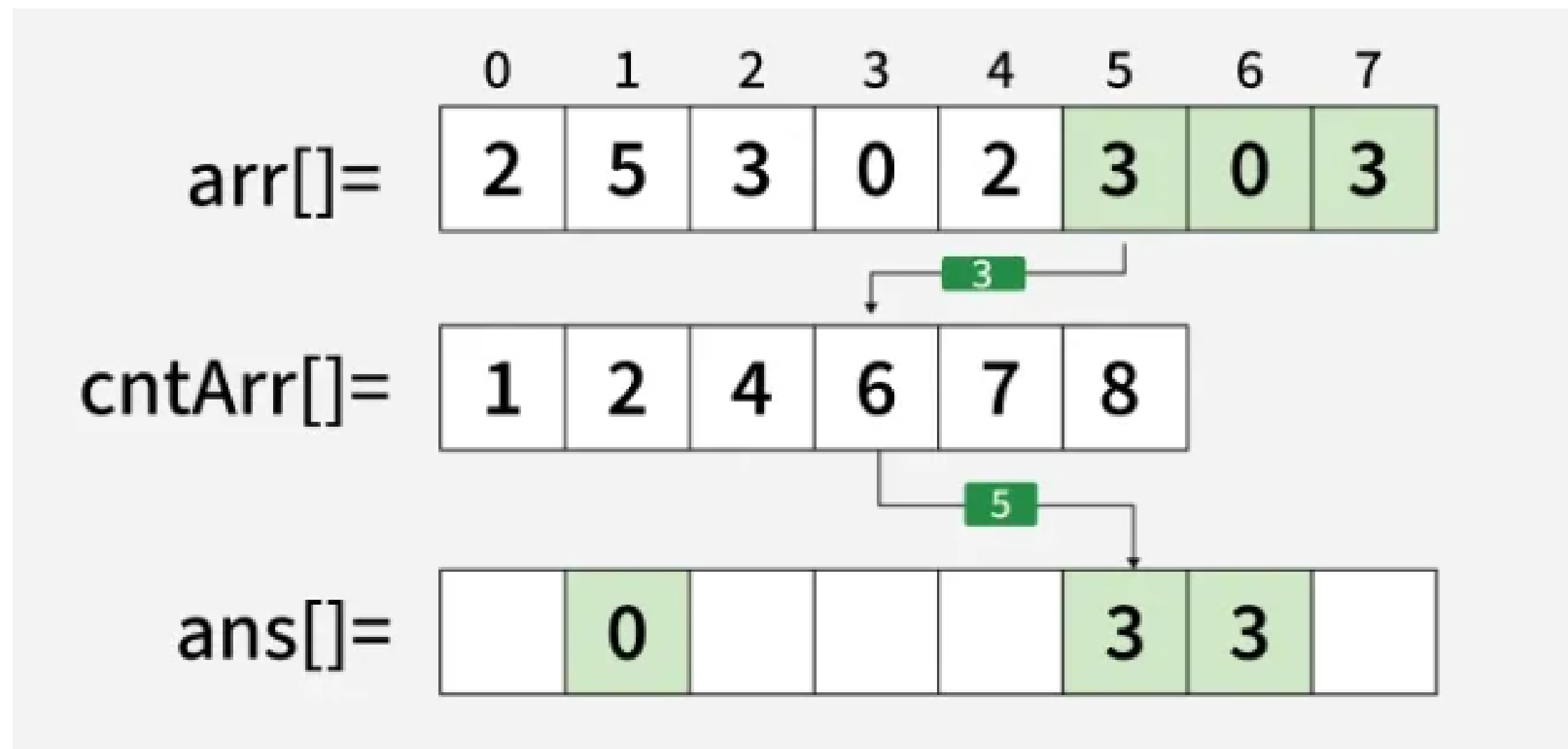


07

Atualizações

Atualizamos resultado $[\text{vetor_contagem} [\text{vetor} [5]] - 1] = \text{vetor} [5]$

também atualizamos $\text{vetor_contagem} [\text{vetor} [5]] = \text{vetor_contagem} [\text{vetor} [5]] --$.

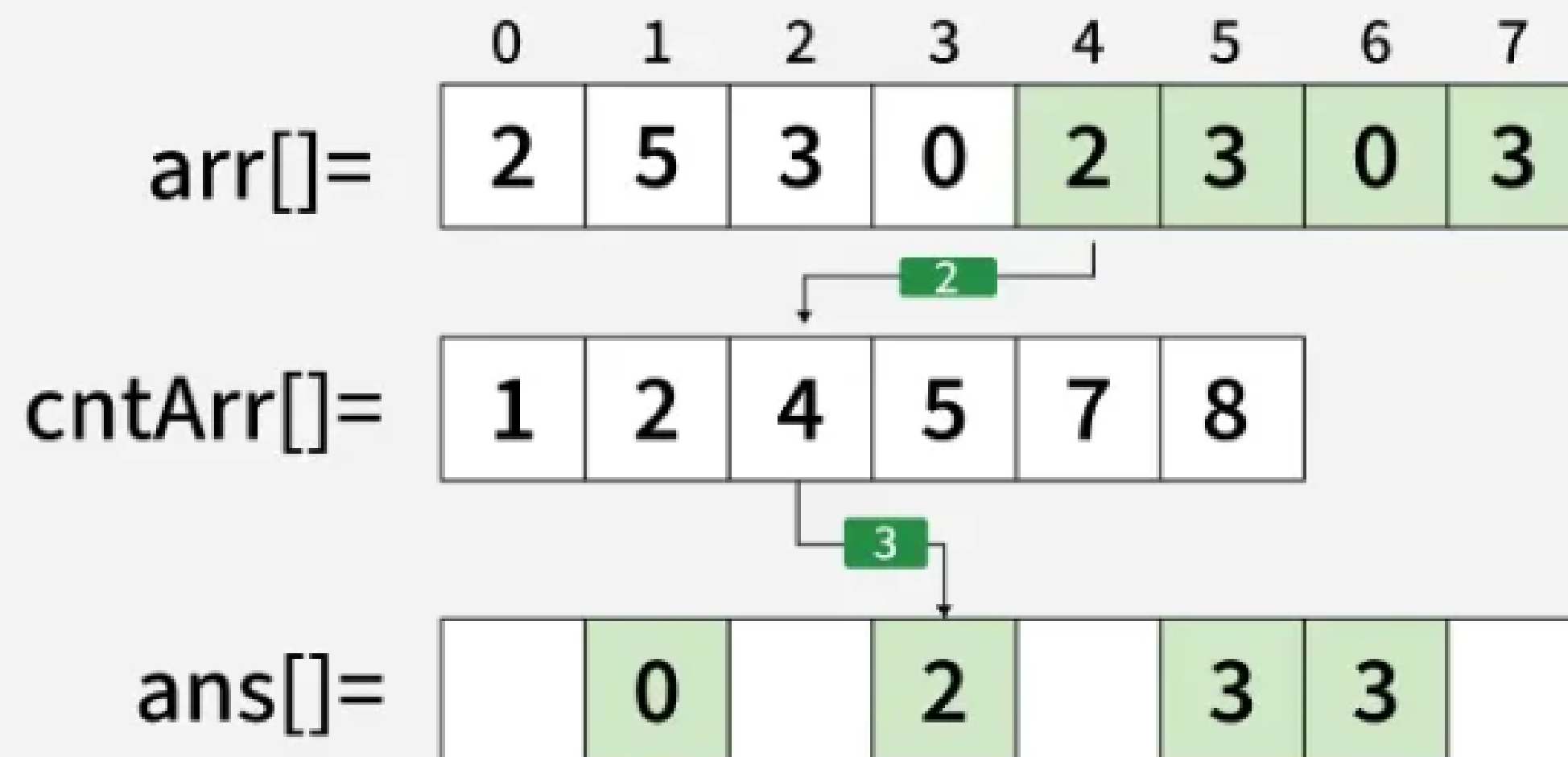


08

Atualizações

Atualizamos resultado $[\text{vetor_contagem} [\text{vetor} [4]] - 1] = \text{vetor} [4]$

também atualizamos $\text{vetor_contagem} [\text{vetor} [4]] = \text{vetor_contagem} [\text{vetor} [4]] --$.

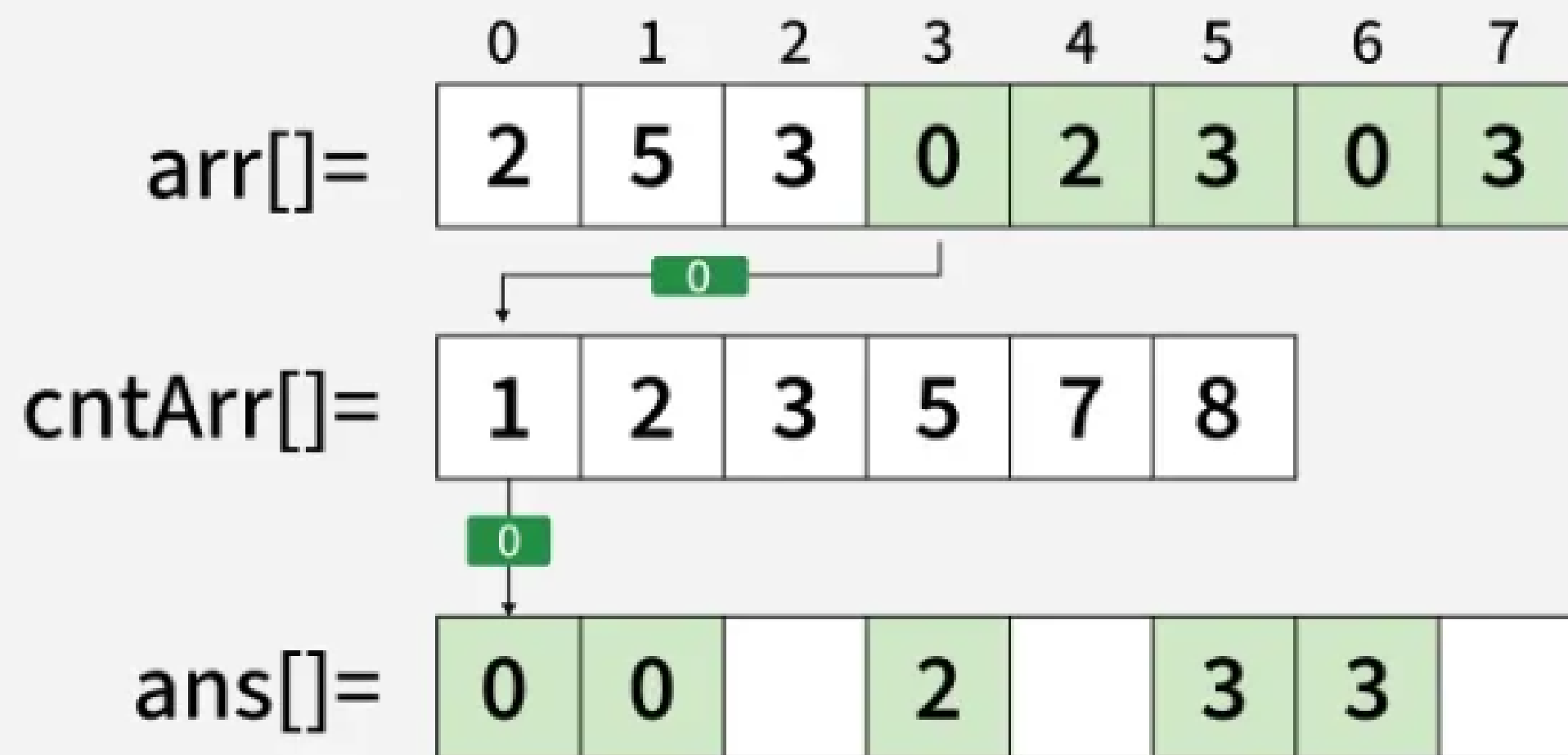


09

Atualizações

Atualizamos resultado $[\text{vetor_contagem} [\text{vetor} [3]] - 1] = \text{vetor} [3]$

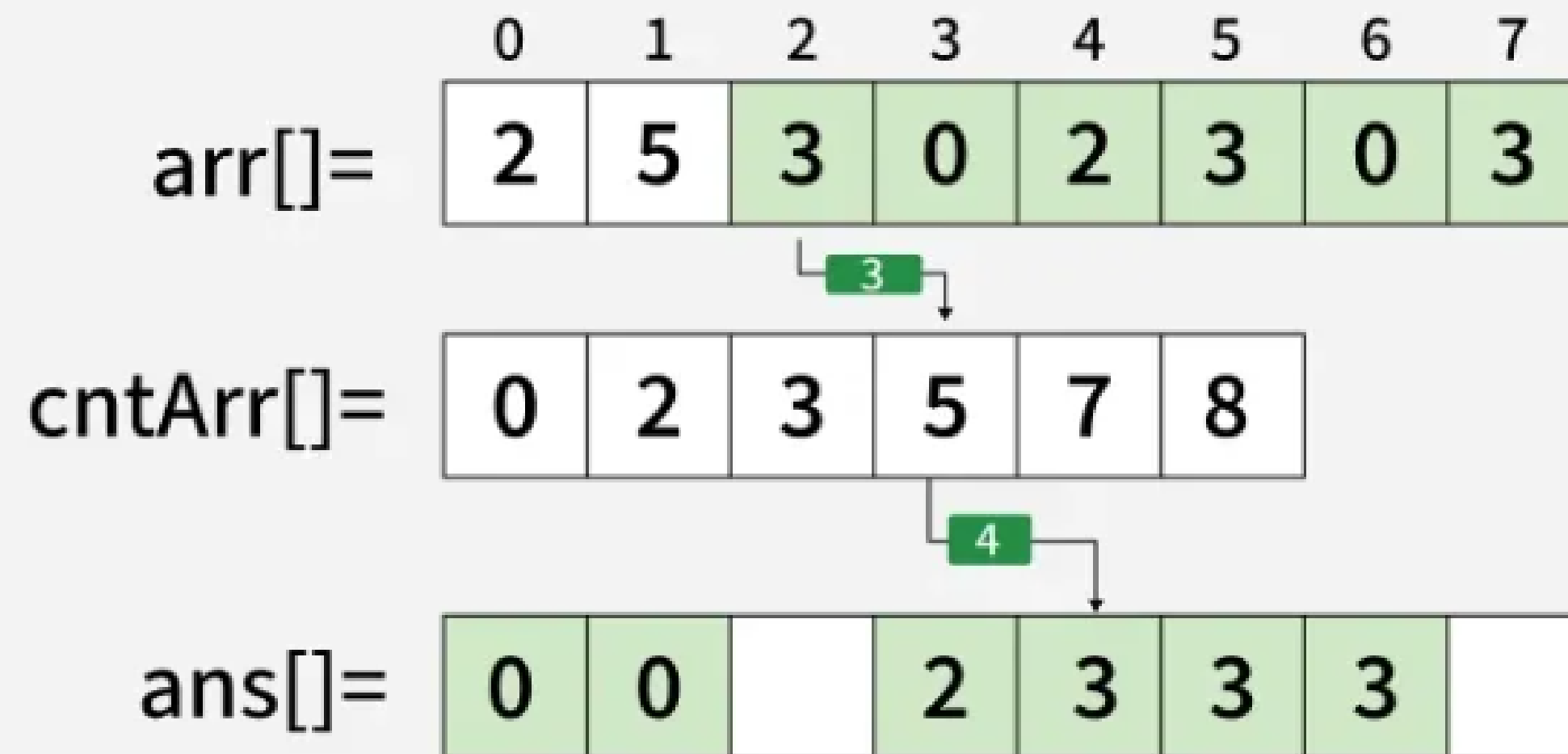
também atualizamos $\text{vetor_contagem} [\text{vetor} [3]] = \text{vetor_contagem} [\text{vetor} [3]] --$.



10 | Atualizações

Atualizamos resultado [vetor_contagem [vetor [2]] - 1] = vetor [2]

também atualizamos vetor_contagem [vetor [2]] = vetor_contagem [vetor [2]] -- .

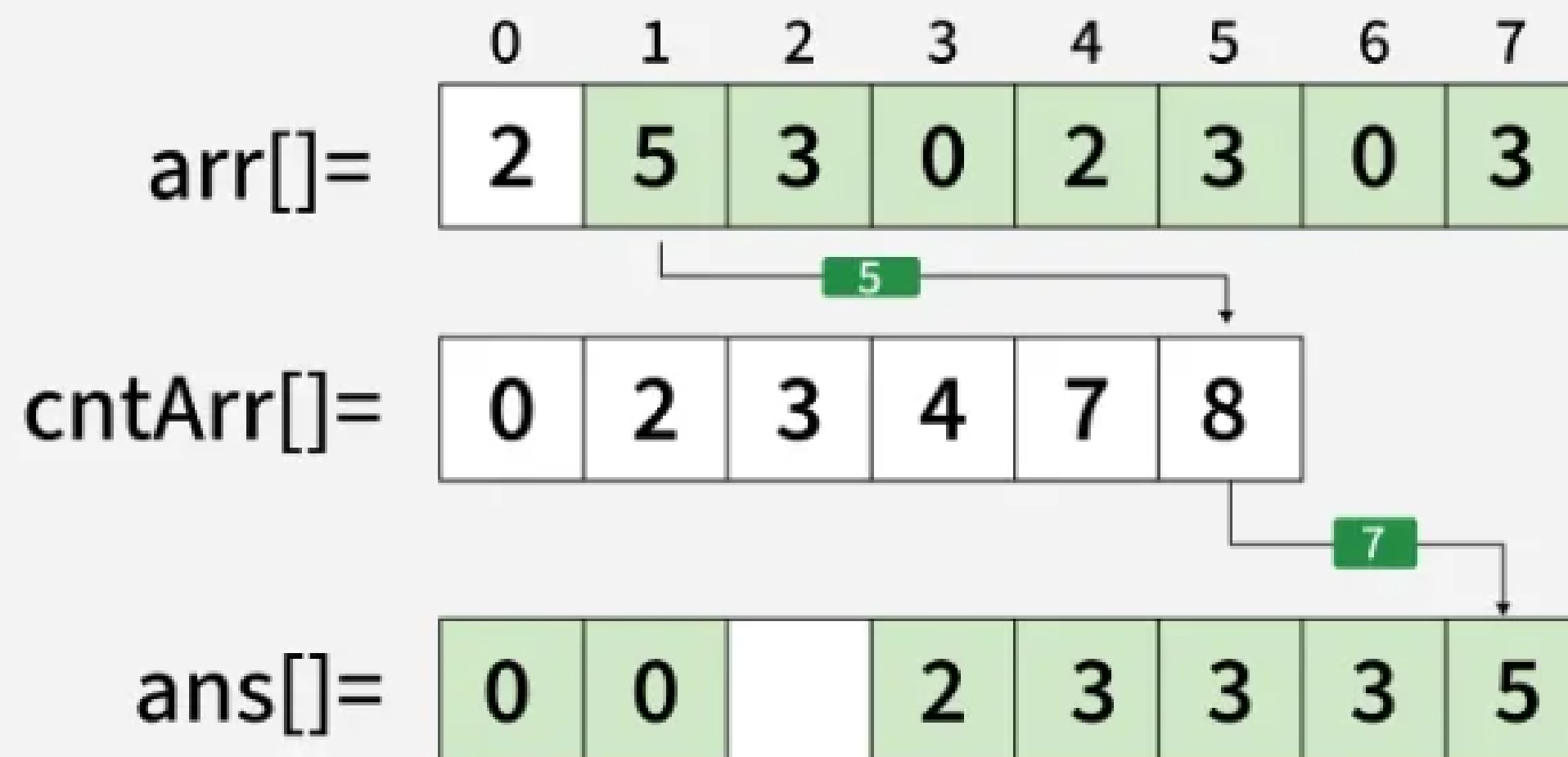


11

Atualizações

Atualizamos resultado $[\text{vetor_contagem} [\text{vetor} [1]] - 1] = \text{vetor} [1]$

também atualizamos $\text{vetor_contagem} [\text{vetor} [1]] = \text{vetor_contagem} [\text{vetor} [1]] --$.

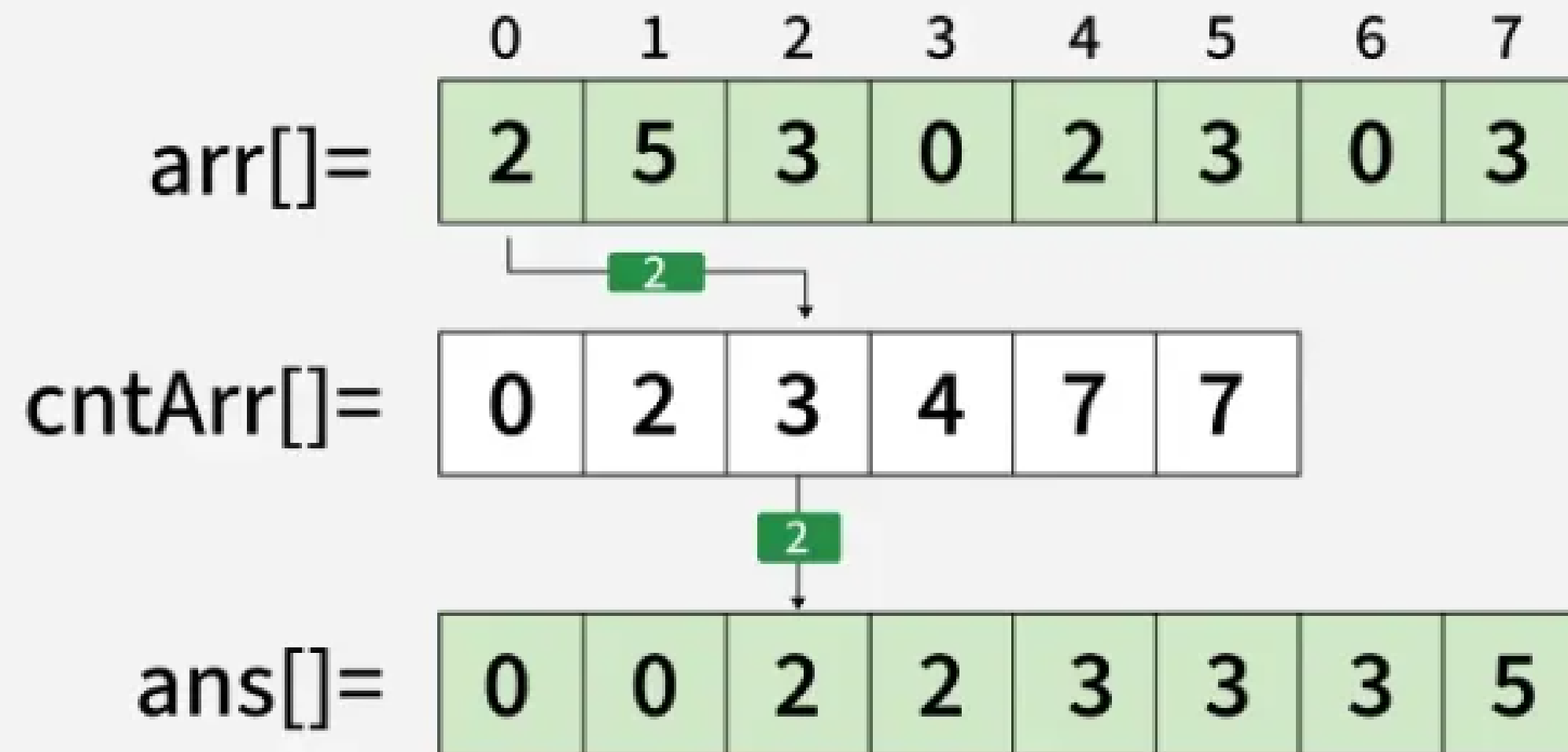


12

Atualizações

Atualizamos resultado [vetor_contagem [vetor [0]] - 1] = vetor [0]

também atualizamos vetor_contagem [vetor [0]] = vetor_contagem [vetor [0]] -- .



Complexidade do Counting Sort

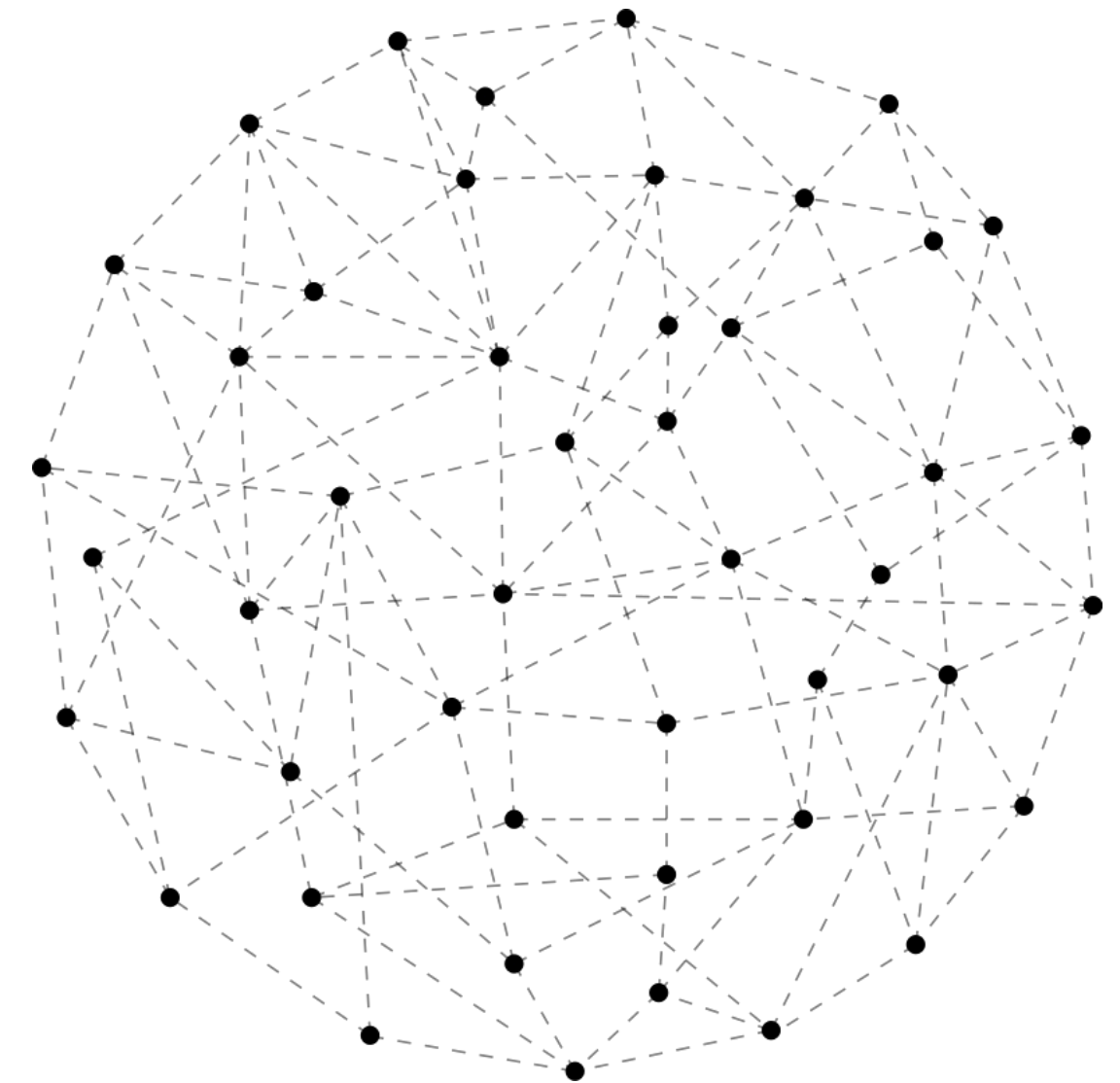
A complexidade do Counting Sort é definida como $O(n + k)$, sendo “n” o tamanho da entrada e “k” a faixa de valores a serem ordenados.

Esse algoritmo é especialmente performático para a ordenação de grandes vetores porém com uma faixa reduzida de valores.

Melhor caso

Pior caso

Caso médio



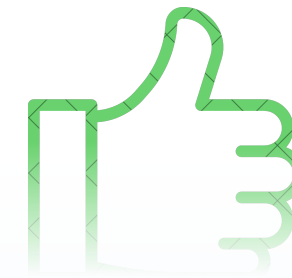
Vantagens

Suas vantagens estão relacionadas principalmente à eficiência que proporcionam na organização de conjuntos de dados, à melhoria no desempenho de algoritmos subsequentes e à possibilidade de adaptação a diferentes contextos computacionais.



**Complexidade
linear**

01



Estabilidade

02



**Não
Comparativo**

03

Desvantagens

Apesar de sua relevância, algoritmos de ordenação apresentam limitações que devem ser analisadas conforme o contexto de aplicação.



Armazenamento

01



Dependente

02



Limitado

03